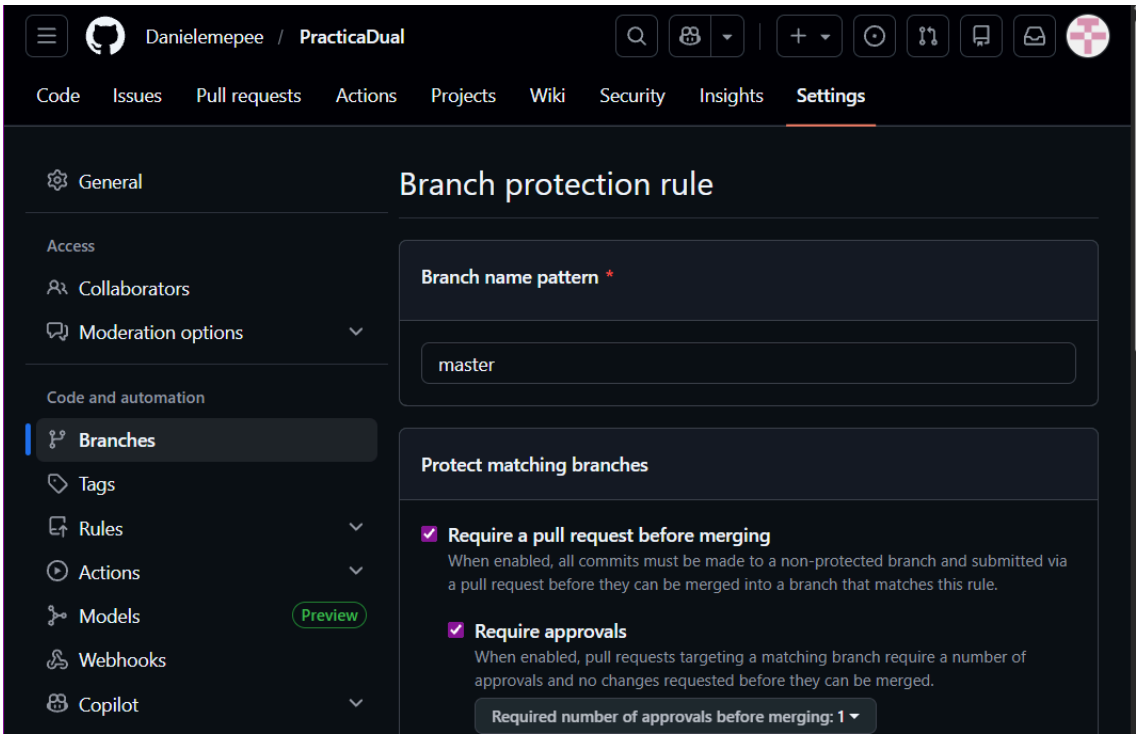
TRABAJO ENTORNO GITHUB

Daniel Martín/ Álamo

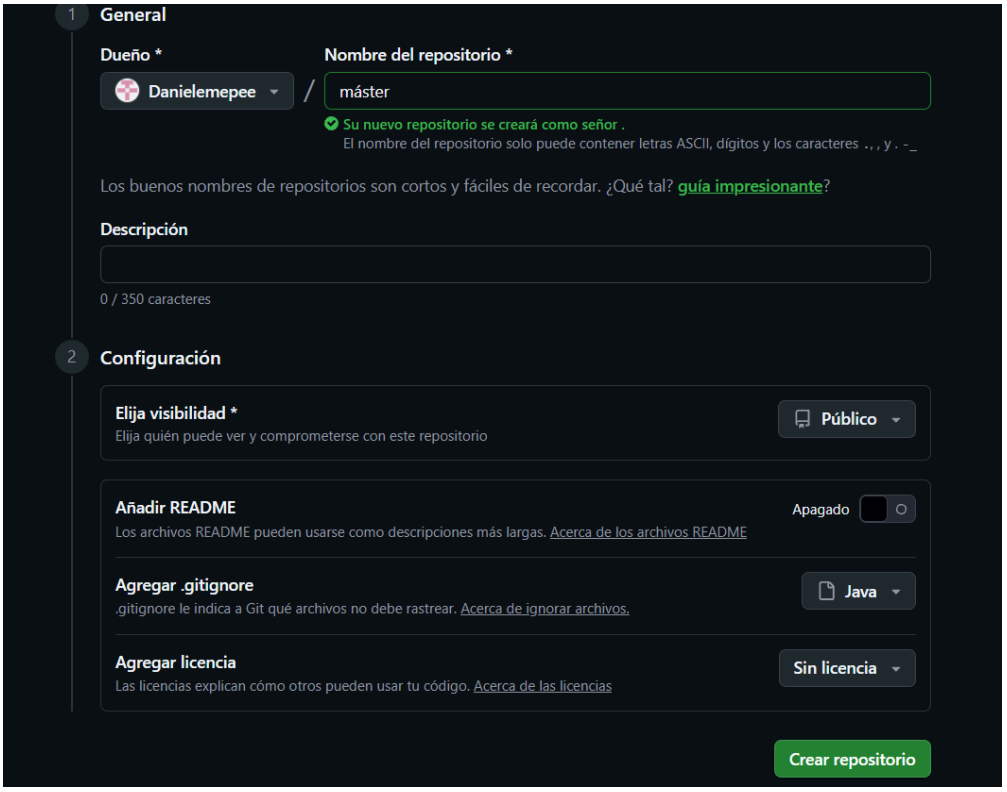
TAREA 1: Repositorio GIT **2**
TAREA 2: Identificación de patrones **8**
TAREA 3: Implementación de la refactorización **9**
1º Refactorización cambiar nombre a las variables 9
2º Refactorización extraer método 10
3º Refactorización implementar getters y setters 12
TAREA 4: Documentación técnica **13**
1º Documentar los métodos con la sintaxis JavaDoc 13
2º Generar la documentación JavaDoc en formato HTML 16
3º Crear el archivo README.md 18

TAREA 1: Repositorio GIT

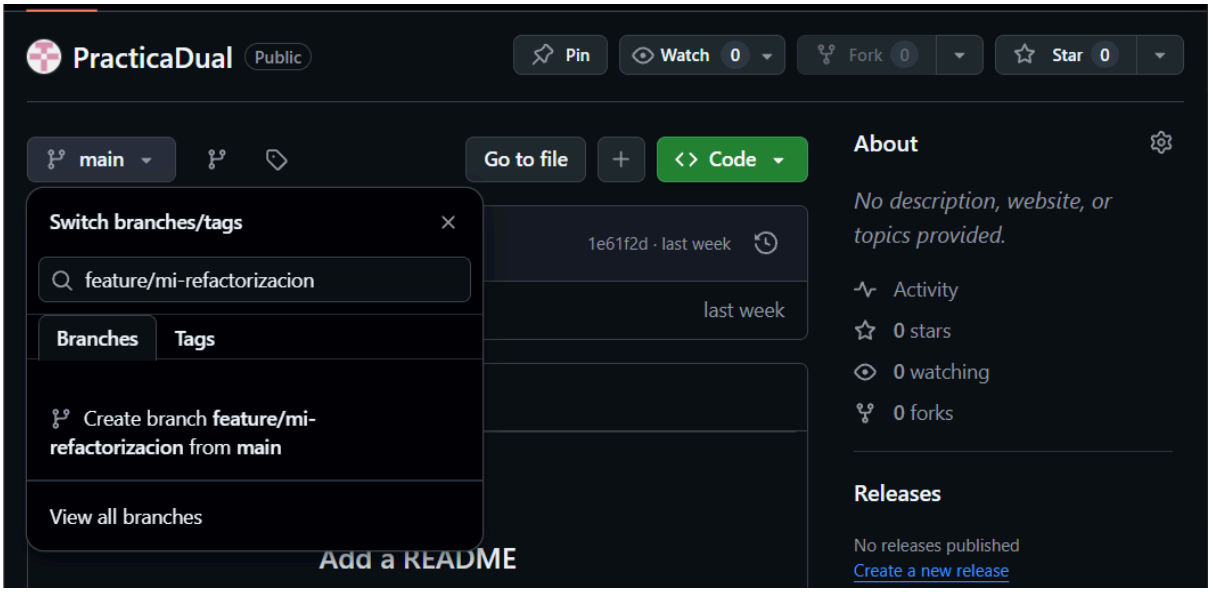
Protección de la rama master



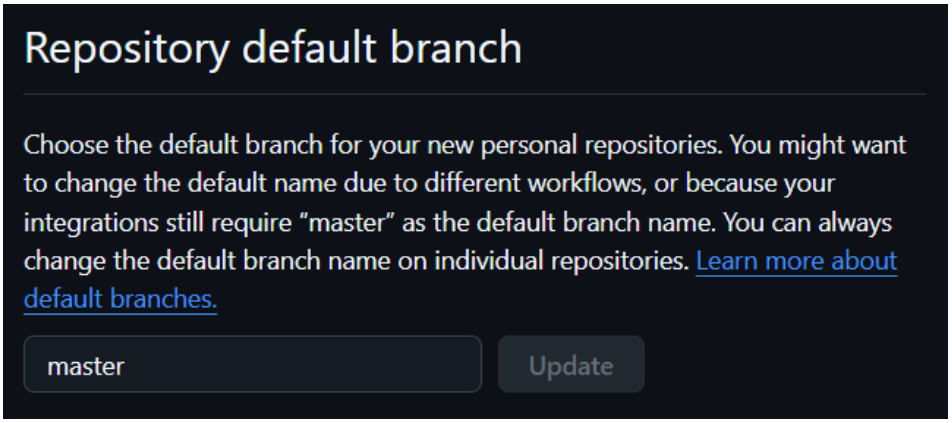
Añadir un .gitignore



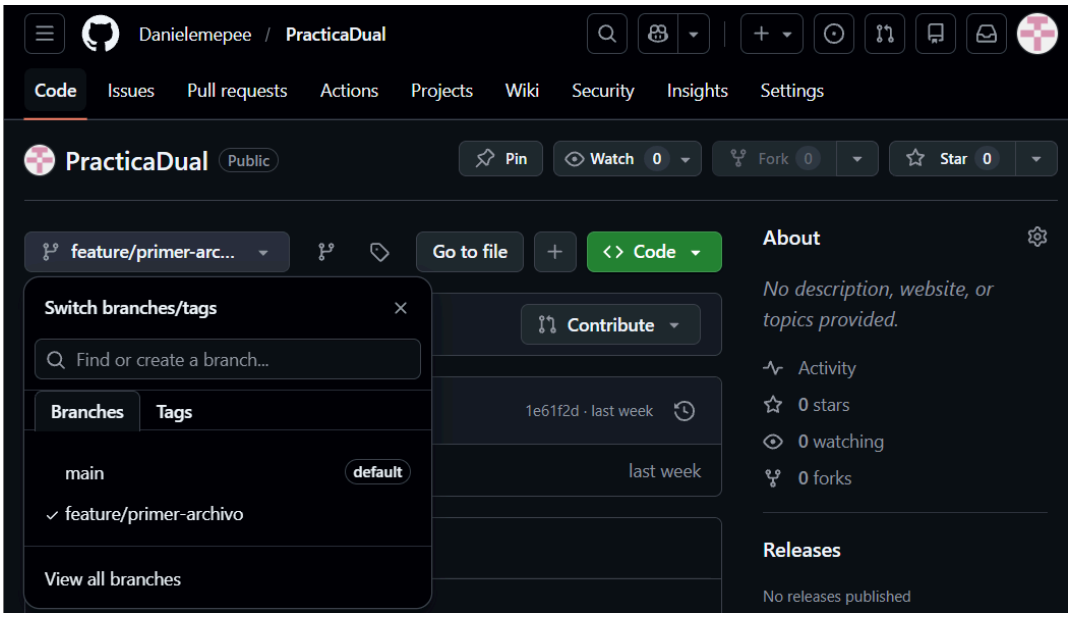
Rama nueva



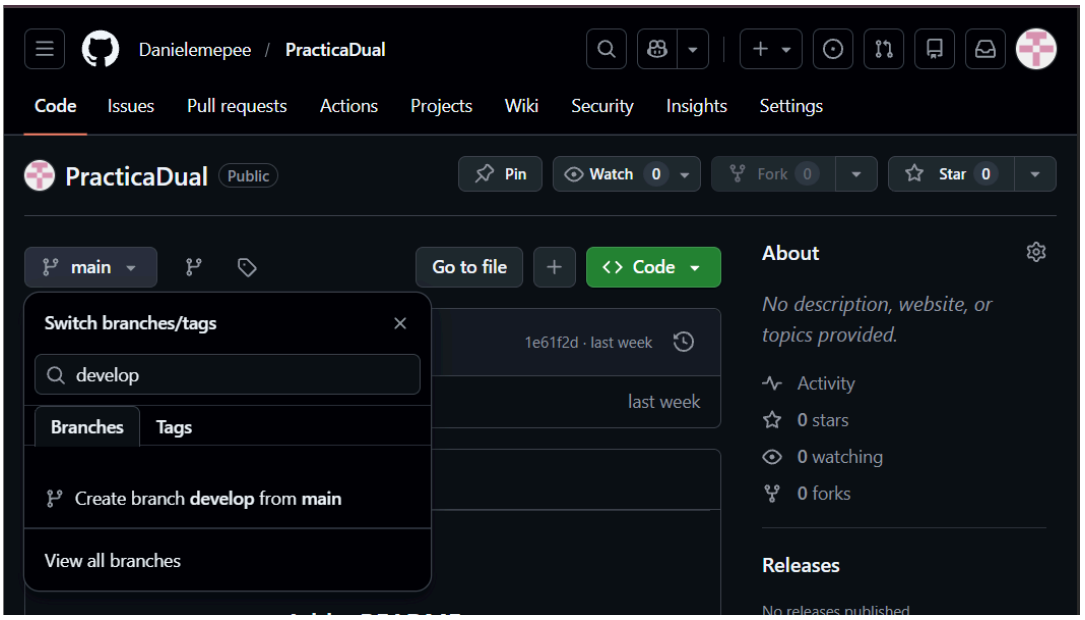
Cambiar nombre a Master



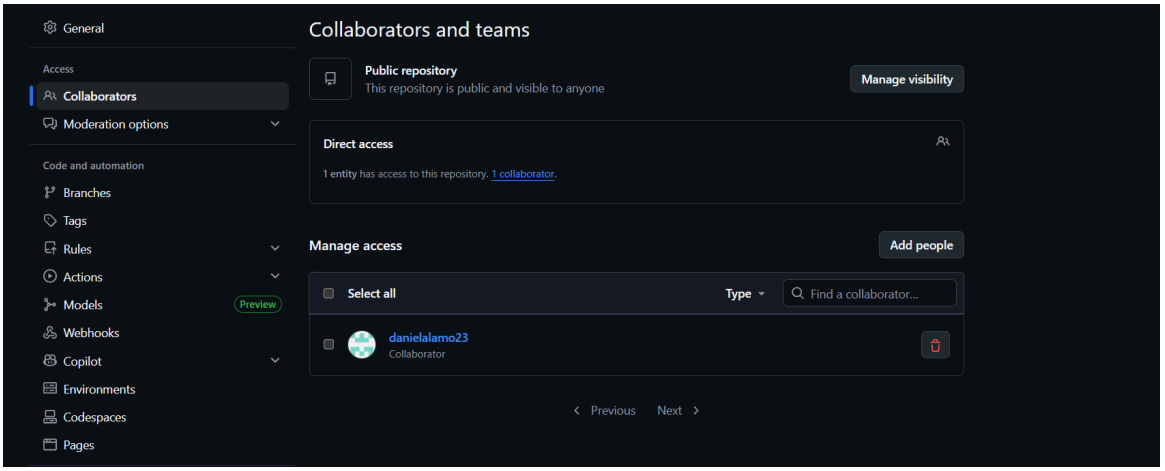
Creación de rama de trabajo



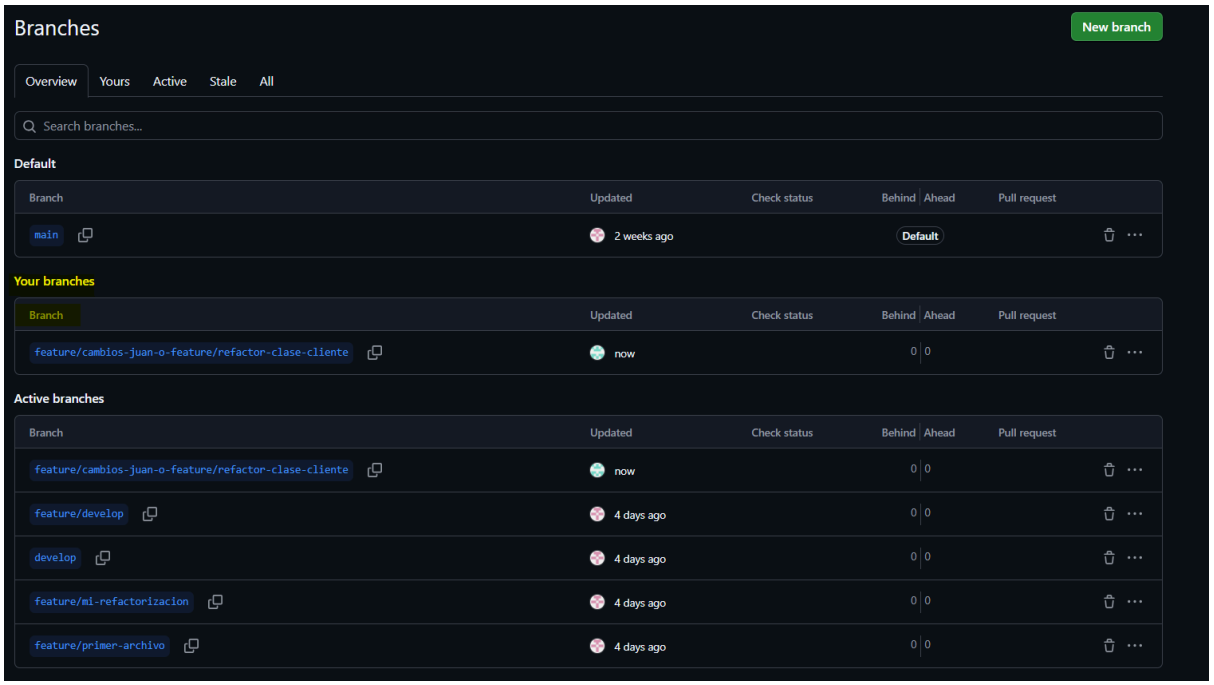
Creo la rama develop



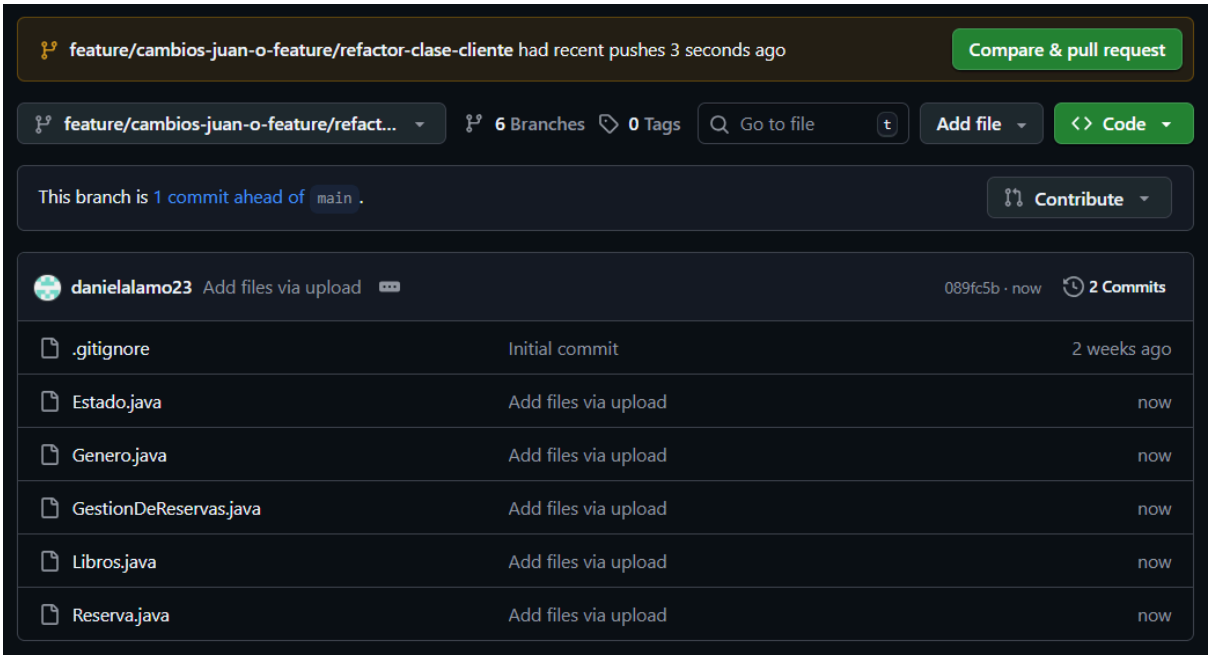
Añado a mi compañero



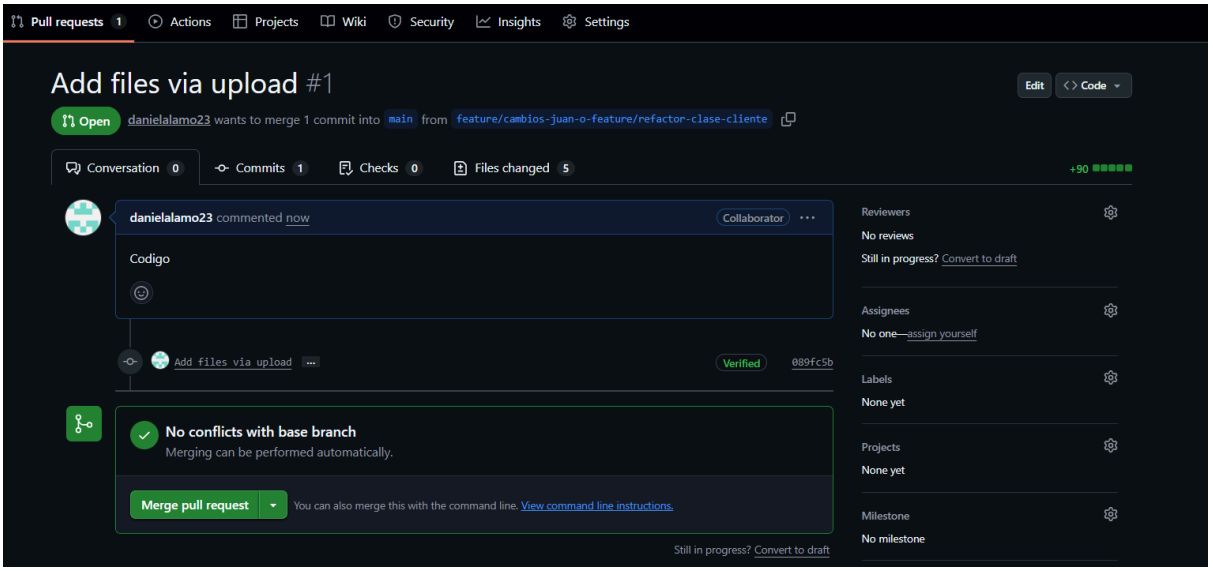
El compañero crea su propia rama



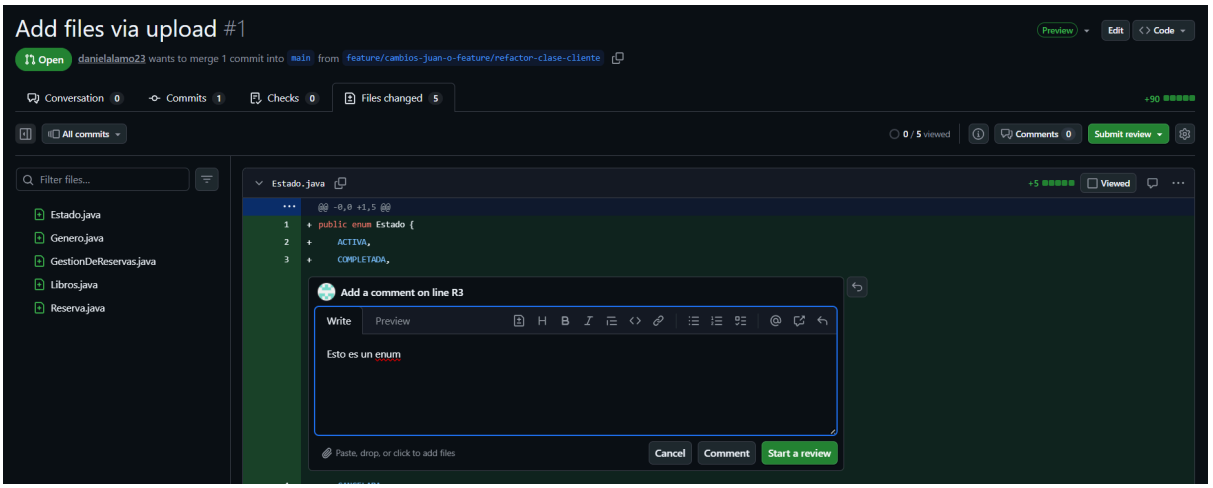
El compañero hacer un cambio (Añadimos código a nuestra rama)



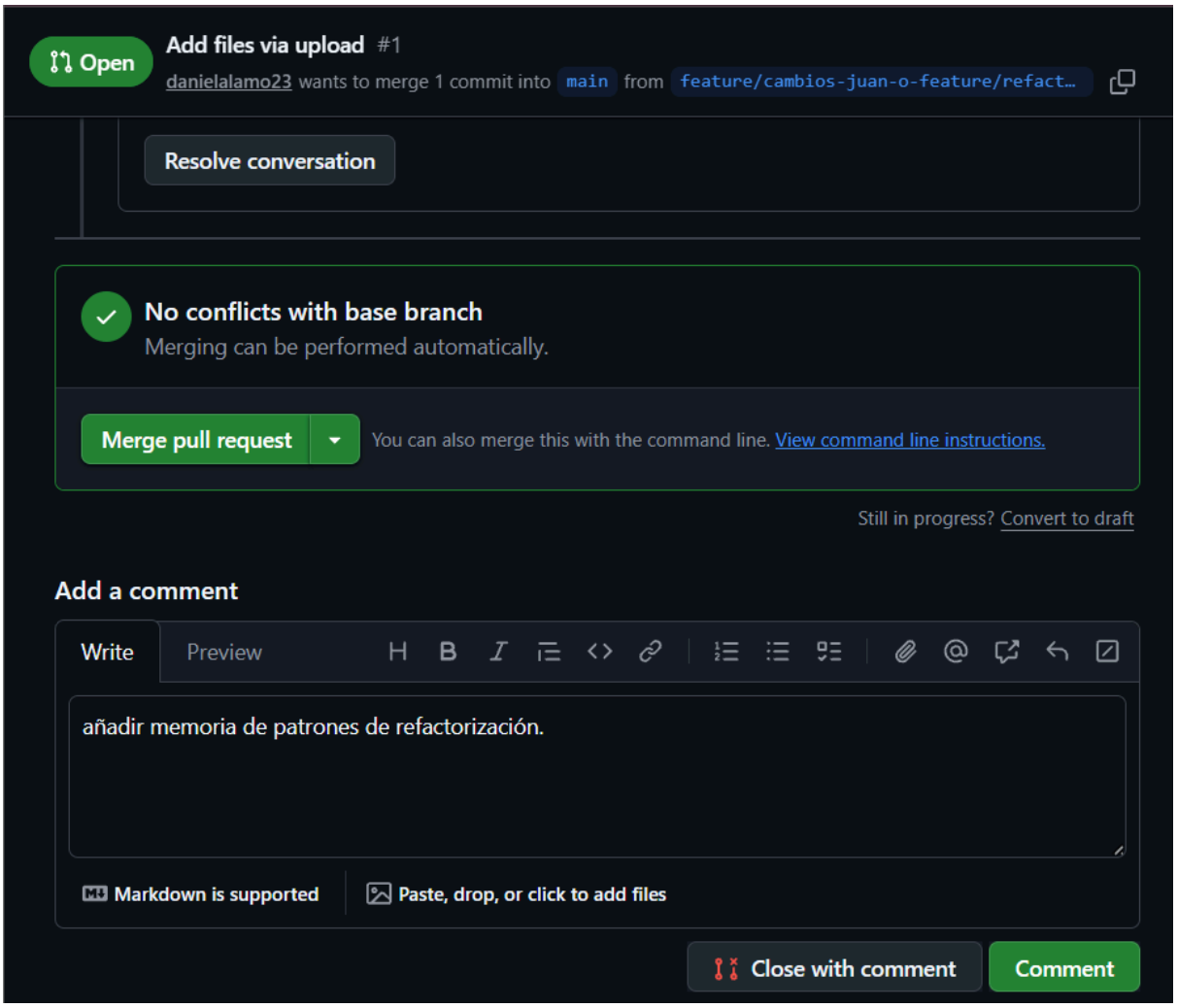
Ahora hacemos un Pull Request



Comentamos dentro del pull request



Creación del documento MEMORIA.md



TAREA 2: Identificación de patrones

```
package com.refactoring.projects.project12;

public class EMailMessage {

    private String remitente;
    private String destinatario;
    private String asunto;
    private String cuerpo;

    public EMailMessage(String remitente, String destinatario, String asunto, String cuerpo) {
        this.remitente = remitente;
        this.destinatario = destinatario;
        this.asunto = asunto;
        this.cuerpo = cuerpo;
    }

    public void enviar() {
        System.out.println("Enviando email de " + remitente + " a " + destinatario);
    }
}
```

Este código actualmente es funcional pero se puede modificar para poder tener características de Clean Code

Análisis de Refactorización

1. EMailMessage (Constructor): Tiene 4 parámetros de tipo String, haciendo que en un futuro el constructor se volvería inestable y a lo mejor puede dar errores de orden

2. Encapsulamiento: Actualmente, los atributos están en privtae, haciendo que no haya forma de acceder a la información en el momento de crear el objeto. Habría que añadir métodos getter

3. La clase EMailMessage tiene 2 trabajos, uno de ellos es definir la estructura del mensaje y ejecuta enviar().

   Se debe de separar el Modelo del Servicio

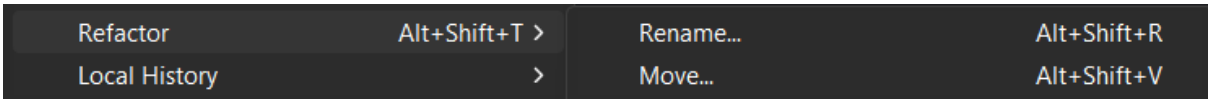
TAREA 3: Implementación de la refactorización

Lo primero que debemos hacer es entrar en el github para ver el código que hay que refactorizar, después iremos al Eclipse y lo importamos:

Código EmailMessage.java SMSMessage.java

1º Refactorización cambiar nombre a las variables

Hacemos clic sobre el nombre del atributo, pulsamos - Alt + Shift + R (atajo de Rename), escribimos el nuevo nombre y enter. Y sin atajos clic derecho Refactor > Rename



EmailMessage.java

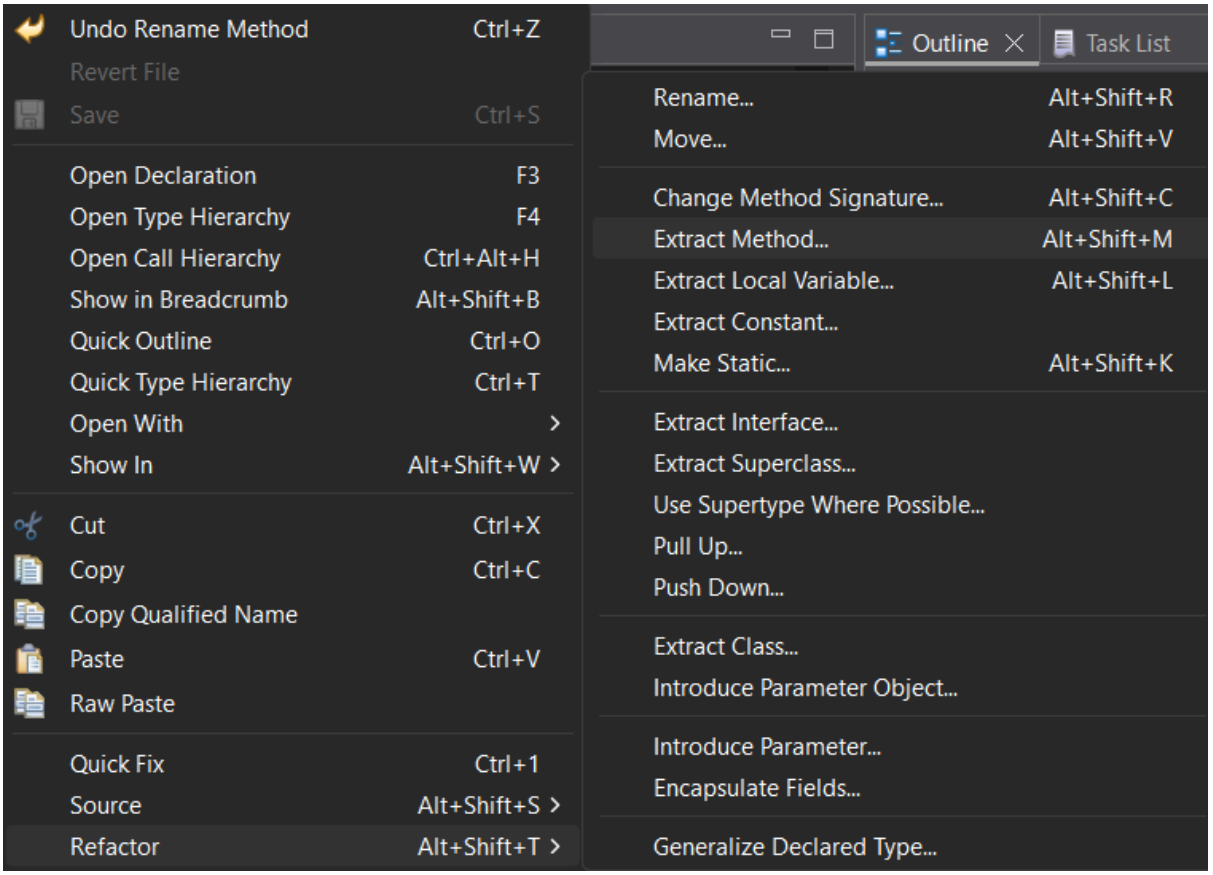
```java
public class EMailMessage {

    private String sender;
    private String recipient;
    private String affair;
    private String body;

    public EMailMessage(String remitente, String destinatario, String asunto, String cuerpo) {
        this.sender = remitente;
        this.recipient = destinatario;
        this.affair = asunto;
        this.body = cuerpo;
    }

    public void send() {
        System.out.println("Enviando email de " + sender + " a " + recipient);
    }
}
```

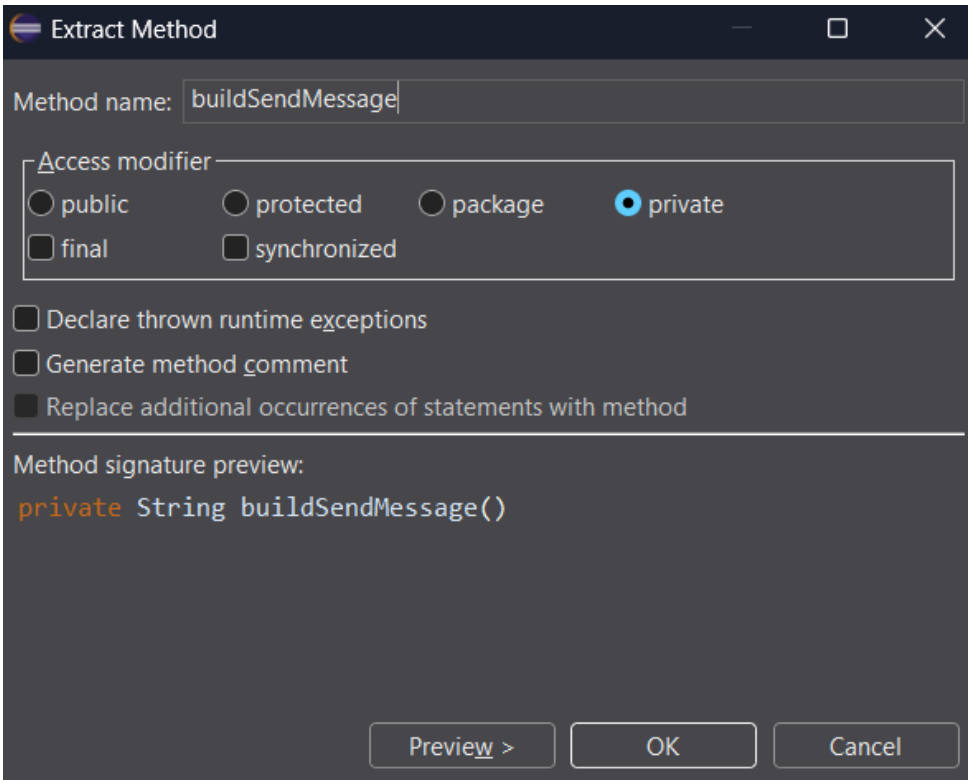
SMSMessage.java

```java
public class SMSMessage {

    private String sender;
    private String recipient;
    private String body;

    public SMSMessage(String remitente, String destinatario, String cuerpo) {
        this.sender = remitente;
        this.recipient = destinatario;
        this.body = cuerpo;
    }

    public void send() {
        System.out.println("Enviando SMS de " + sender + " a " + recipient);
    }
}
```

2º Refactorización extraer método

Para esto lo que vamos a hacer es seleccionar la parte del código del mensaje - Clic derecho > Refactor > Extract Method y le ponemos un nombre





SMSMessage.java

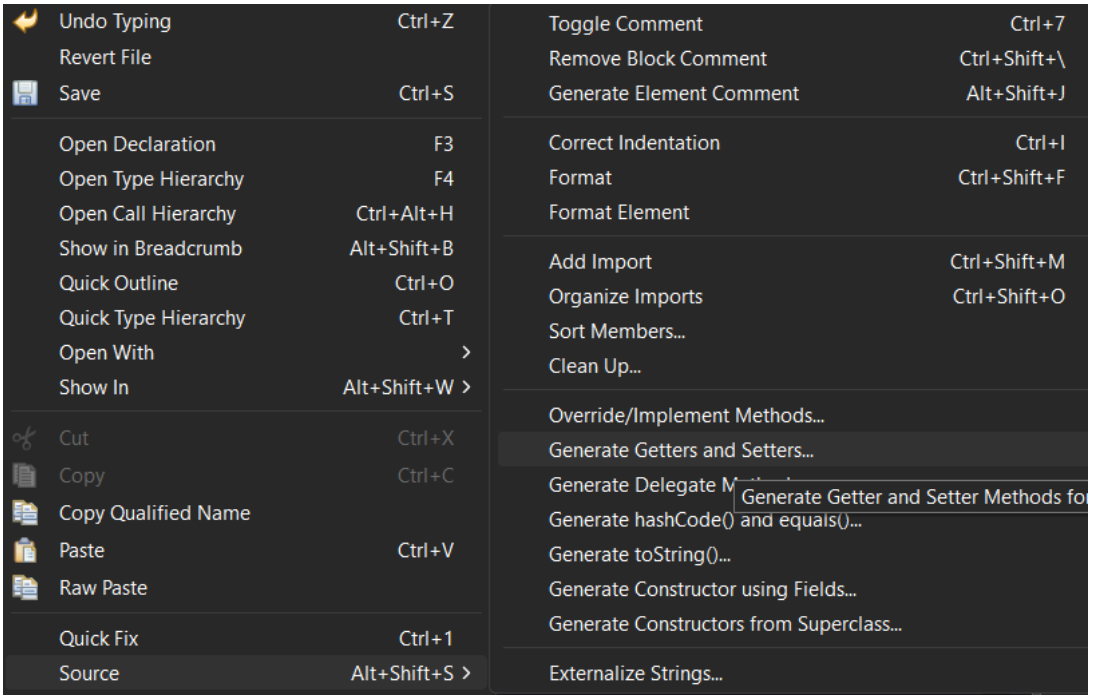
```java
public class SMSMessage {

    private String sender;
    private String recipient;
    private String body;

    public SMSMessage(String remitente, String destinatario, String cuerpo) {
        this.sender = remitente;
        this.recipient = destinatario;
        this.body = cuerpo;
    }

    public void send() {
        System.out.println(buildSendMessage());
    }

    private String buildSendMessage() {
        return "Enviando SMS de " + sender + " a " + recipient;
    }
}
```

EmailMessage.java

```java
public class EMailMessage {

    private String sender;
    private String recipient;
    private String affair;
    private String body;

    public EMailMessage(String remitente, String destinatario, String asunto, String cuerpo) {
        this.sender = remitente;
        this.recipient = destinatario;
        this.affair = asunto;
        this.body = cuerpo;
    }

    public void send() {
        System.out.println(buildSendMessage());
    }

    private String buildSendMessage() {
        return "Enviando email de " + sender + " a " + recipient;
    }
}
```

3º Refactorización implementar getters y setters

Clic derecho dentro de la clase, Source > Generate Getters and Setters y seleccionamos los atributos.



EmailMessage.java

```java
public String getSender() {
    return sender;
}

public void setSender(String sender) {
    this.sender = sender;
}

public String getRecipient() {
    return recipient;
}

public void setRecipient(String recipient) {
    this.recipient = recipient;
}

public String getAffair() {
    return affair;
}

public void setAffair(String affair) {
    this.affair = affair;
}

public String getBody() {
    return body;
}

public void setBody(String body) {
    this.body = body;
}
```

SMSMessage.java

```java
public String getSender() {
    return sender;
}

public void setSender(String sender) {
    this.sender = sender;
}

public String getRecipient() {
    return recipient;
}

public void setRecipient(String recipient) {
    this.recipient = recipient;
}

public String getBody() {
    return body;
}

public void setBody(String body) {
    this.body = body;
}
```

TAREA 4: Documentación técnica

1º Documentar los métodos con la sintaxis JavaDoc

En estas capturas en **EMailMessage.java** aparece affair como subject porque me equivoque en la traducción

En **SMSMessage.java**, encima de public class **SMSMessage {** escribimos:

```
/**
 * Represents an SMS message with a sender, a recipient and a text body.
 */

public class SMSMessage {
```

En **EMailMessage.java**:

```
/**
 * Represents an email message with sender, recipient, subject and body.
 */

public class EMailMessage {
```

En **SMSMessage.java**, encima del constructor:

```
/**
 * Creates a new SMS message.
 *
 * @param sender the phone number of the sender
 * @param recipient the phone number of the recipient
 * @param body the text content of the SMS
 */

public SMSMessage(String remitente, String destinatario, String cuerpo) {
    this.sender = remitente;
    this.recipient = destinatario;
    this.body = cuerpo;
}
```

En **EMailMessage.java**:

```
/**
 * Creates a new email message.
 *
 * @param sender the email address of the sender
 * @param recipient the email address of the recipient
 * @param subject the subject line of the email
 * @param body the text content of the email
 */

public EMailMessage(String remitente, String destinatario, String asunto, String cuerpo) {
    this.sender = remitente;
    this.recipient = destinatario;
    this.subject = asunto;
    this.body = cuerpo;
}
```

En **SMSMessage.java,** encima del método **send():**

```java
/**
 * Sends the message by printing a description to the console.
 */
public void send() {
    System.out.println(buildSendMessage());
}
```

En **EMailMessage.java**:

```java
/**
 * Sends the message by printing a description to the console.
 */

public void send() {
    System.out.println(buildSendMessage());
}
```

En **SMSMessage.java,** encima del método **buildSendMessage():**

```java
/**
 * Builds the message that will be printed when sending this SMS.
 *
 * @return a formatted description of the sending action
 */

private String buildSendMessage() {
    return "Enviando SMS de " + sender + " a " + recipient;
}
```

En **EMailMessage.java**:

```java
/**
 * Builds the message that will be printed when sending this Email.
 *
 * @return a formatted description of the sending action
 */

private String buildSendMessage() {
    return "Enviando email de " + sender + " a " + recipient;
}
```

En **SMSMessage.java** para los getters y setters:

```java
/**
 * Returns the sender of the SMS.
 *
 * @return the sender phone number
 */
public String getSender() {
    return sender;
}

/**
 * Sets the sender of the SMS.
 *
 * @param sender the new sender phone number
 */
public void setSender(String sender) {
    this.sender = sender;
}

/**
 * Returns the recipient of the SMS.
 *
 * @return the recipient phone number
 */
public String getRecipient() {
    return recipient;
}

/**
 * Sets the recipient of the SMS.
 *
 * @param recipient the new recipient phone number
 */
public void setRecipient(String recipient) {
    this.recipient = recipient;
}
```

```java
/**
 * Returns the body of the SMS.
 *
 * @return the SMS text content
 */
public String getBody() {
    return body;
}

/**
 * Sets the body of the SMS.
 *
 * @param body the new SMS text content
 */
public void setBody(String body) {
    this.body = body;
}
```

En **EMailMessage.java** para los getters y setters:

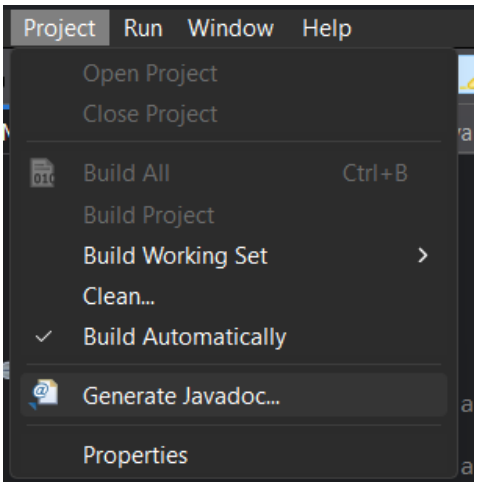
```java
/**
 * Returns the sender of the email.
 *
 * @return the sender email address
 */
public String getSender() {
    return sender;
}

/**
 * Sets the sender of the email.
 *
 * @param sender the new sender email address
 */
public void setSender(String sender) {
    this.sender = sender;
}

/**
 * Returns the recipient of the email.
 *
 * @return the recipient email address
 */
public String getRecipient() {
    return recipient;
}

/**
 * Sets the recipient of the email.
 *
 * @param recipient the new recipient email address
 */
public void setRecipient(String recipient) {
    this.recipient = recipient;
}
```

```java
/**
 * Returns the subject of the email.
 *
 * @return the subject line
 */
public String getSubject() {
    return subject;
}

/**
 * Sets the subject of the email.
 *
 * @param subject the new subject line
 */
public void setSubject(String subject) {
    this.subject = subject;
}

/**
 * Returns the body of the email.
 *
 * @return the email text content
 */
public String getBody() {
    return body;
}

/**
 * Sets the body of the email.
 *
 * @param body the new email text content
 */
public void setBody(String body) {
    this.body = body;
}
```

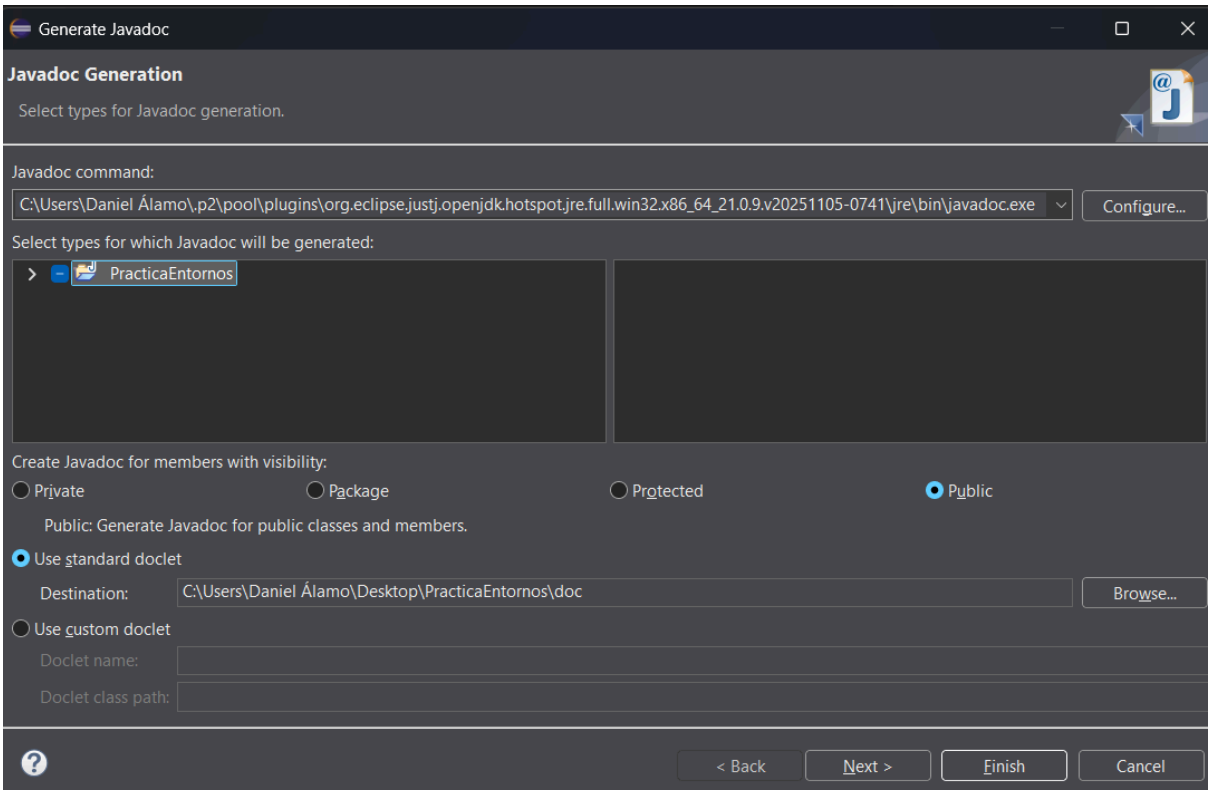
2º Generar la documentación JavaDoc en formato HTML

Para hacer el doc tendremos que seguir los siguientes pasos:

- Menú superior > Project
- Seleccionar Generate Javadoc



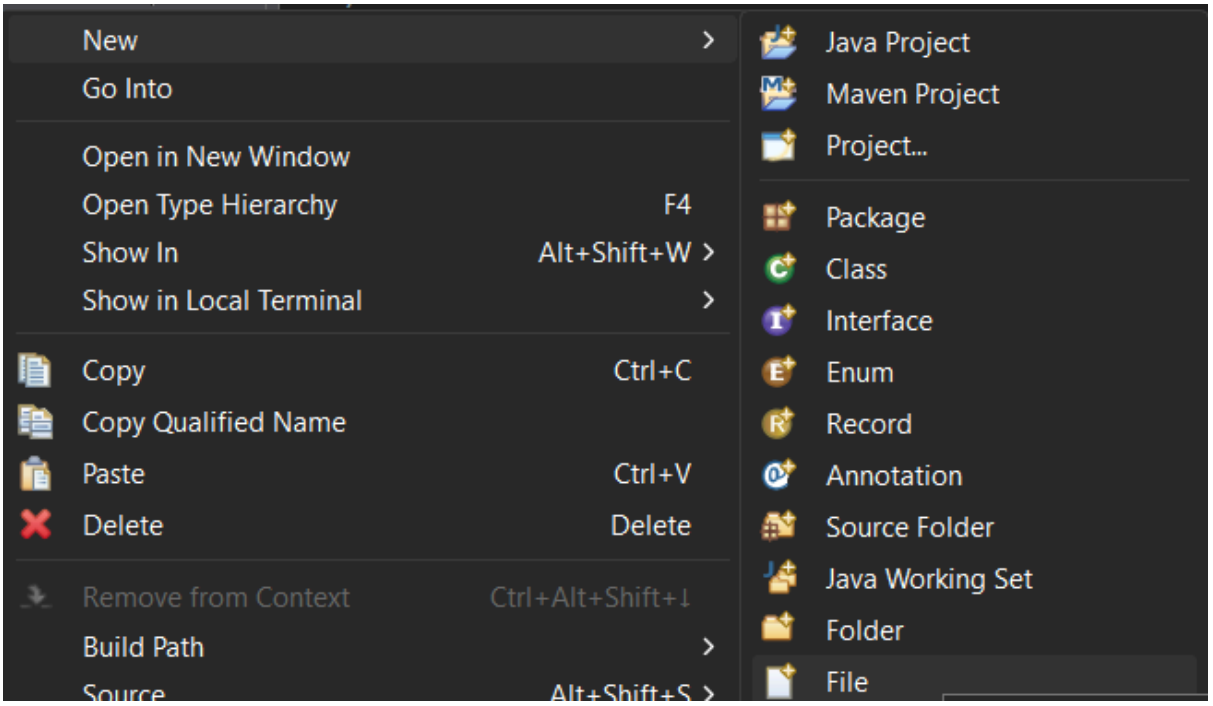
- En la ventana emergente:
  - Seleccionar el proyecto
  - Asegurarse de que la carpeta está marcada
- Elegir la carpeta de salida: PracticaEntornos/doc
- Pulsar Finish



Tuve que cambiar los nombres en los constructores(de español a inglés) por un fallo y quedaron asi:

```
public SMSMessage(String sender, String recipient, String body) {
    this.sender = sender;
    this.recipient = recipient;
    this.body = body;
}
```

```
public EMailMessage(String sender, String recipient, String subject, String body) {
    this.sender = sender;
    this.recipient = recipient;
    this.subject = subject;
    this.body = body;
}
```

3º Crear el archivo README.md

En el panel izquierdo hacemos clic derecho sobre nuestro proyecto, seleccionamos new > file y después le asignamos el nombre **README.md**